

Bluefin vessel modelling workflow: from a simple Python model to a MATLAB-inspired nonlinear model

Purpose of this document

This document explains, in plain language, the modelling workflow that was followed to build a practical simulation model of the Bluefin vessel from experiment logs. It is written as a baseline guide for future work, especially for readers with little or no background in hydrodynamics or ship modelling.

The document covers four main ideas:

1. how the original simple Python model worked, and why it could not reproduce the real vessel well;
2. how the MATLAB model was interpreted and translated into a Python model with richer physics;
3. how the comparison and sweep scripts were designed and used to fit the simulator to the real-vessel tests; and
4. how the second version of the MATLAB-inspired Python model (`ship_model_bluefin_matlab_style_v2.py`) improved the fit.

The main message is that **modelling a vessel from experiment logs is an iterative system-identification process**. You do not write one model, tune one parameter, and finish. Instead, you:

- choose a model structure;
- extract measurable benchmarks from experiments;
- compare model outputs against those benchmarks;
- identify what kind of error remains;
- improve the model structure or the parameters; and
- repeat.

1. Why a vessel model is needed

Reinforcement learning (RL) depends heavily on the environment in which the agent is trained. If the simulator gives the wrong relationship between:

- throttle and speed,
- rudder and turn rate,
- speed and turning radius,
- acceleration and distance travelled,

then the agent learns the wrong physical world.

For a surface vessel, the most important low-level motions in calm water are usually:

- **surge**: forward/backward motion,
- **sway**: sideways motion,
- **yaw**: rotation in heading.

A model that only approximates forward motion and heading very roughly can still be useful for early debugging, but it is usually not accurate enough for reliable sim-to-real RL transfer.

2. Basic concepts used in vessel modelling

Before discussing the code, it helps to define a few terms.

2.1 Body frame vs world frame

A vessel has two useful coordinate systems.

- The **world frame** describes position on the map, usually as (x, y) .
- The **body frame** is attached to the vessel itself.

In the body frame:

- u is the forward velocity (surge),
- v is the sideways velocity (sway),
- r is the yaw rate (rotation rate).

The body-frame variables are the most natural way to describe vessel dynamics, because thrust, drag, rudder force, and inertia act relative to the vessel.

2.2 Kinematics vs dynamics

- **Kinematics** tells us how velocities produce position and heading changes.
- **Dynamics** tells us how forces and moments produce accelerations.

For example:

- thrust and drag affect $\dot{u}$,
- lateral forces affect $\dot{v}$,
- turning moments affect $\dot{r}$,
- then u, v, r produce $\dot{x}, \dot{y}, \dot{\psi}$.

2.3 Why boats are harder than cars

A car mostly points where it moves. A vessel does not always do that. A ship can:

- move forward,
- drift sideways,
- rotate,
- and feel strong fluid-memory effects from the water around it.

That is why sway and yaw dynamics matter so much.

2.4 Added mass and damping

A vessel accelerating in water must accelerate some of the surrounding water too. This is often represented using **added mass**.

Also, water produces resistance that depends on velocity, often nonlinearly. This is represented by **hydrodynamic damping**.

3. The original simple Python model

The original `ship_model.py` was intentionally simple. It used:

- one forward speed state v ,
- one heading state h ,
- one yaw-rate state w ,
- one quadratic drag term,
- one lumped thrust law,
- one lumped yaw damping term.

Its force and moment calculation can be summarized as:

$$T = K_T \cdot rpm^2$$

$$\delta = \delta_{\max} \cdot \frac{rud}{100}$$

$$X = T \cos \delta - C_D v^2$$

$$N = T \sin \delta \cdot L_r - C_N w$$

where:

- T is thrust,
- δ is rudder angle,
- X is forward force,
- N is yaw moment,
- C_D is the drag coefficient,
- C_N is the yaw damping coefficient,
- L_r is an effective rudder offset or moment arm.

Then acceleration was computed as:

$$\dot{v} = \frac{X}{m}$$

and yaw acceleration was computed as:

$$\dot{w} = \frac{N}{I_z}$$

with the moment of inertia approximated as:

$$I_z \approx \frac{1}{2}mL_r^2$$

3.1 Why this model was useful at first

The old model was:

- easy to understand,
- fast to run,
- easy to connect to a Gym/RL environment,
- good for very early prototyping.

It was a reasonable first model because it provided the correct *qualitative* control inputs:

- more throttle gives more forward motion,
- more rudder gives more turning.

3.2 Why the old model could not match the real vessel

The old model could not match the real Bluefin well because it ignored too much of the vessel physics.

(a) No sway state The old model had no explicit sideways velocity v in the ship-dynamics sense. That means it could not represent:

- side-slip in turning,
- lateral hull force balance,
- coupling between sway and yaw.

(b) No added mass The old model used a simple physical mass but did not model added mass. So the transient response was too crude.

(c) Rudder model too simple The old model assumed turning comes directly from thrust redirected by rudder angle:

$$N \propto T \sin \delta$$

In reality, a rudder creates a force because it interacts with the local water flow. That force depends on:

- the forward flow speed,
- sway,
- yaw rate,
- rudder geometry,
- rudder angle,
- hull-rudder interaction.

So the old model had no good way to reproduce:

- realistic turning onset,
- realistic yaw-rate growth,
- realistic speed loss in turning,
- realistic turn radius at different speeds.

(d) One drag term could not fit both early and late motion A single drag term of the form $C_D v^2$ can sometimes fit top speed, but it usually cannot fit:

- low-speed acceleration,
- medium-speed distance buildup,
- and high-speed saturation,

all at the same time.

(e) No actuator dynamics The old model assumed the rudder and thrust commands were applied immediately. Real actuators have limits and delays:

- rudder rate limits,
- motor/propeller spool dynamics,
- sensor and control delays.

3.3 Summary of the limitation

The old Python model was a **lumped low-order control-oriented model**. It was suitable for early debugging, but it did not have enough degrees of freedom to reproduce the real vessel's surge-sway-yaw behaviour.

4. The real-vessel log analysis workflow

To improve the model, two real-vessel tests were used.

4.1 Speed test

This test provided benchmarks such as:

- peak forward speed,
- acceleration over selected windows,
- distance travelled after 5 s and 10 s,
- time to 50% and 90% of peak speed.

4.2 Turning test

This test provided benchmarks such as:

- peak yaw rate,
- time to reach 30°, 60°, 90°, and 180° heading change,
- speed during the turn,
- radius estimated from the first 90° and first 180° of turning.

4.3 Why metrics were extracted instead of comparing only trajectory plots

Trajectory plots are useful visually, but they do not tell you *which part* of the model is wrong.

For example, two trajectories can look similar even when one has the wrong:

- acceleration profile,
- yaw rate,
- or turn-speed retention.

So the logs were converted into **metrics** that isolate different physical effects.

4.4 Important lesson from the logs

The direct replay of RC signals $S1/S2$ was not reliable enough, because the meaning of the controller values was uncertain. That is why the workflow eventually moved from **input-replay fitting** to **output-only fitting**.

This is an important general lesson:

If control inputs are uncertain or not well calibrated, calibrate the model against measured motion outputs instead.

5. Reading the MATLAB Bluefin model

The MATLAB model was much richer than the old Python model. It contained the typical structure of a **nonlinear 3-DOF manoeuvring model**.

The intended state variables were essentially:

- surge velocity u ,
- sway velocity v ,
- yaw rate r ,
- heading ψ ,
- rudder angle δ ,
- world position (x, y) .

The model computed forces and moments from three sources:

$$\begin{aligned}X &= X_H + X_P + X_R \\Y &= Y_H + Y_P + Y_R \\N &= N_H + N_P + N_R\end{aligned}$$

where:

- H = hull contribution,
- P = propeller contribution,
- R = rudder contribution.

This is a standard and very useful decomposition.

5.1 Hull contribution

The hull terms include resistance and coupling effects such as:

- surge drag,
- sway damping,
- yaw damping,
- sway–yaw coupling,
- nonlinear terms such as $|v|v$, $|r|r$, and cross terms.

This is why the MATLAB model can represent turning far more realistically than the old Python model.

5.2 Propeller contribution

The propeller contributes surge force, primarily controlled by rpm. In the MATLAB script, the propeller force is more physically structured than in the old Python model.

5.3 Rudder contribution

The rudder creates a normal force from the local inflow, then resolves that into:

- a longitudinal component,
- a lateral component,
- and a yaw moment.

The generic structure is:

$$F_N = \frac{1}{2}\rho A_R C_L U_R^2 \sin(\alpha_R)$$

and then

$$\begin{aligned}X_R &= -(1 - t_R)F_N \sin \delta \\Y_R &= -(1 + a_H)F_N \cos \delta \\N_R &= -(x_R + a_H x_H)F_N \cos \delta\end{aligned}$$

This is much more realistic than a single $T \sin \delta$ turning moment.

5.4 Rudder actuator dynamics

The MATLAB model also limited:

- maximum rudder angle,
- maximum rudder rate.

This is important because real steering does not jump instantly to the commanded angle.

5.5 What was learned from the MATLAB model

The MATLAB model taught three major lessons:

1. surge, sway, and yaw must be modelled together;
2. hull, propeller, and rudder should be represented as separate physical components; and
3. actuator dynamics matter.

6. Translating MATLAB into Python: the first MATLAB-inspired Python model

The file `ship_model_bluefin_matlab_style.py` was not a literal line-by-line transcription of the MATLAB code. Instead, it was a **structural translation**.

That means the Python version preserved the important ideas:

- 3-DOF body dynamics,
- added mass,
- nonlinear hull damping,
- propeller force,
- rudder force and yaw moment,
- rudder saturation and rate limiting,
- transformation from body motion to world motion,

while adapting them to a stable Python implementation and to the old simulator interface.

6.1 Why a literal line-by-line port was not ideal

The MATLAB code as shared had some inconsistencies between comments, indexing, and non-dimensional variables. It looked like a research code or teaching code rather than a clean drop-in simulation module.

So the correct engineering choice was:

- keep the **physics structure**,
- but rewrite the implementation in a **clean dimensional form**.

6.2 State vector used in Python

The Python model used a state vector like:

$$[u, v, r, \psi, \delta, x, y]$$

where:

- u = surge speed,
- v = sway speed,
- r = yaw rate,
- ψ = heading,
- δ = actual rudder angle,
- x, y = world coordinates.

6.3 Effective inertia

Instead of using a crude yaw inertia based only on mass and a lever arm, the Python model used:

$$m_{11} = m + m_x$$

$$m_{22} = m + m_y$$

$$m_{33} = I_z + J_z$$

This is much closer to a real marine craft model.

6.4 Kinematics

The Python version used body-to-world kinematics of the form:

$$\begin{aligned}\dot{x} &= u \sin \psi - v \cos \psi \\ \dot{y} &= u \cos \psi + v \sin \psi \\ \dot{\psi} &= r\end{aligned}$$

using the heading convention already used in the old project.

6.5 Numerical integration

The Python model used **RK4** (Runge–Kutta 4th order integration), rather than explicit Euler, to improve stability.

6.6 Why this first MATLAB-style Python model was better than the old one

It could represent:

- sideways motion in turning,
- yaw-rate dynamics more realistically,
- rudder-limited steering response,
- hull-rudder interaction more realistically,
- and more realistic coupling between surge and turn.

However, after calibration attempts, it still showed important mismatch patterns.

7. Why the first MATLAB-style Python model still needed improvement

After running sweeps and comparisons against the real vessel, a clear error pattern appeared.

7.1 Surge pattern

The model tended to be:

- too weak at low speed,
- but too strong at higher speed,

or equivalently:

- too slow to build speed,
- but still able to overshoot the real peak speed in some configurations.

This means the propeller/thrust model still lacked one important feature: **the thrust should vary with vessel speed**, not just rpm.

7.2 Turn pattern

The turn fit showed that a single rudder-force scaling parameter was not enough.

The model could sometimes match:

- turn timing,
- or turn radius,

but not both at the same time as:

- peak yaw rate,
- and speed preserved during the turn.

That meant the rudder model needed more flexibility.

8. The role of the sweep scripts

The sweep files were written to automate calibration. Instead of guessing one parameter at a time manually, the scripts:

1. selected a grid of candidate parameter values,
2. ran the simulation for each combination,
3. extracted the same metrics as the real-vessel logs,
4. compared simulated metrics to real metrics, and
5. ranked the parameter sets by total error.

8.1 Why sweeps were necessary

A vessel model has many interacting parameters. Changing one parameter often affects multiple behaviours at the same time. For example:

- changing thrust affects both peak speed and turn authority;
- changing drag affects both top speed and turn-speed retention;
- changing yaw damping affects both peak yaw rate and turn timing.

So a sweep is a practical way to explore the trade-offs.

8.2 Relative-error comparison

Each script used errors of the form:

$$\text{relative error} = \frac{|y_{sim} - y_{real}|}{|y_{real}|}$$

and a weighted score of the form:

$$J = \sum_i w_i e_i$$

where:

- e_i is a relative error for one metric,
- w_i is its importance weight.

8.3 First sweep family: replay-based comparison

The earliest sweep scripts tried to replay the control histories (**S1**, **S2**) from the real logs into the simulator.

This was useful conceptually, but it failed in practice because:

- the exact meaning of **S1** and **S2** was not known well enough;
- neutral and full-forward values were inferred imperfectly;
- startup periods and controller artefacts distorted the replay;
- the simulator could spend most of the replay using unrealistic inputs.

This is a very valuable lesson:

If the actuator mapping is uncertain, input-replay fitting may give misleading results.

8.4 Output-only comparison

Because replay-based fitting was unreliable, the workflow moved to **output-only sweeps**.

This means:

- for surge, use constant-rpm straight runs,
- for turning, use constant-rpm constant-rudder runs,
- then match the resulting motion to the real motion metrics.

This was a much better strategy, because it removed the uncertain controller mapping from the fitting problem.

8.5 Separate surge and turn sweeps

Two separate sweeps were built:

- a **surge sweep** for forward-speed behaviour,
- a **turn sweep** for turning behaviour.

This was important because the physics of straight motion and turning are related but not identical.

8.6 Joint sweep

After finding good regions for surge and turning separately, a **joint sweep** was used to find a compromise parameter set that worked reasonably well for both tests.

This is standard in system identification:

- fit each subsystem first,
 - then fit the shared model.
-

9. The second model: `ship_model_bluefin_matlab_style_v2.py`

The second version of the model was created because the first MATLAB-inspired model still lacked enough flexibility to match the real-vessel surge and turn responses simultaneously.

The key idea in v2 was **not** to throw away the physics. Instead, v2 kept the same basic 3-DOF framework but added a few carefully chosen extra terms to correct the specific error patterns observed in the sweeps.

9.1 Improvement 1: speed-shaped thrust

In the first MATLAB-style Python model, propeller thrust was still too simple. In v2, the thrust law was modified to include:

- a **low-speed boost**, and
- a **high-speed decay**.

A simplified form of the v2 thrust model is:

$$X_P \propto rpm |rpm| \cdot \underbrace{(1 + Be^{-u/U_0})}_{\text{low-speed boost}} \cdot \underbrace{\frac{1}{1 + Cu^2}}_{\text{high-speed decay}}$$

where:

- B controls how much extra thrust is given at low speed,
- U_0 controls how quickly that boost disappears,
- C controls how fast thrust effectiveness decreases with speed.

Why this helps This helps solve a common modelling problem:

- constant thrust laws are often too weak at low speed and too strong at high speed;
- the real propeller-hull system does not behave like a constant gain.

In simple terms:

- when the boat is nearly at rest, the model gives it extra help to get moving;
- when the boat is already moving fast, thrust effect is reduced so top speed does not become unrealistically high.

9.2 Improvement 2: splitting rudder lift, yaw authority, and rudder drag

In earlier versions, one rudder-force scaling factor affected too many things at once.

In v2, the rudder contribution was split into multiple tunable effects:

- RUDDER_FORCE_SCALE – overall rudder lift magnitude,
- RUDDER_YAW_SCALE – how much rudder force contributes to yaw moment,
- RUDDER_X_DRAG_SCALE – how much rudder force bleeds forward speed,
- LINEAR_YAW_DAMP – background yaw damping from the hull / water.

A simplified structure is:

$$F_N \propto k_{lift} \sin(\alpha_R)$$

$$X_R = -k_{drag} |F_N| |\sin \delta|$$

$$Y_R \propto F_N \cos \delta$$

$$N_R \propto k_{yaw} F_N \cos \delta$$

Why this helps Without this split, increasing rudder force can accidentally do all of these at once:

- increase turning too much,
- increase speed loss too much,
- distort the turn radius.

By separating them, v2 can:

- increase yaw authority without losing as much speed,
- or preserve speed without making the turn unrealistically weak.

9.3 Improvement 3: keeping the 3-DOF structure while adding practical fitting flexibility

V2 still keeps the important physics:

- surge–sway–yaw states,
- added mass,

- rudder rate limit,
- hull, propeller, rudder decomposition,
- body-to-world kinematics.

The improvements were therefore **targeted additions**, not a completely different model.

10. What the v2 sweeps showed

The v2 sweeps produced the current best shared model:

- MASS = 64.55
- THRUST_COEF = 0.06
- DRAG_COEF = 1.5
- THRUST_LOW_SPEED_BOOST = 1.6
- THRUST_HIGH_SPEED_DECAY = 0.26
- LINEAR_SURGE_DAMP = 2.0
- TURN_COEF = 3.0
- RUDDER_FORCE_SCALE = 0.32
- RUDDER_YAW_SCALE = 2.6
- RUDDER_X_DRAG_SCALE = 0.02
- LINEAR_YAW_DAMP = 1.5

with best joint score about **6.55**.

10.1 What this model now matches well

For the speed test, it now matches quite well:

- peak forward speed,
- 0–2 s average acceleration,
- distance at 10 s.

10.2 What is still not perfect

It still has clear turning mismatch:

- peak yaw rate is still below the real vessel,
- early turning is too aggressive,
- late turning is still not fully correct,
- the full transient shape of surge is not perfect.

That means the model is **much better**, but still not a perfect replica.

11. A general method for modelling a vessel from experiment logs

The process used here can be generalized into a practical workflow.

Step 1. Start with a model that is as simple as possible, but no simpler

A very simple model is useful initially because it helps you understand the role of each parameter. But it should still include enough states to represent the behaviour you care about.

For surface vessels in calm water, a very strong baseline is:

- surge,
- sway,
- yaw,
- heading,
- rudder state,
- x-y position.

Step 2. Choose physical benchmarks from experiments

Do not fit only to a path plot. Extract metrics such as:

- peak speed,
- acceleration,
- distance after a fixed time,
- peak yaw rate,
- time to 90° / 180° turn,
- turn radius.

Step 3. Fit one subsystem at a time first

For example:

- straight test for surge,
- turning test for yaw/rudder.

Step 4. Identify the *pattern* of mismatch

Instead of only asking “is the score low?”, ask:

- is early acceleration too weak but top speed too high?
- is yaw rate too low but radius too small?
- is speed loss in turning too large?

These patterns tell you which part of the model structure is wrong.

Step 5. Only add new parameters when the error pattern justifies them

Do not add complexity randomly. Each new parameter or equation should solve a clear modelling limitation.

For example:

- low-speed thrust boost was added because the model was too weak early;
- high-speed decay was added because the model could otherwise become too fast later;
- separate rudder yaw and drag scales were added because one rudder gain could not match both turn strength and turn speed.

Step 6. Refit and validate again

Every structural change should be followed by a new sweep and a new comparison.

Step 7. Accept that modelling is iterative

A good engineering model is often built by repeated cycles of:

- model,
- compare,
- diagnose,
- improve.

That is normal.

12. Why similar procedures are used in the literature

The procedure followed here is consistent with well-known marine-modelling practice:

- build a structured manoeuvring model,
- separate hull, propeller, and rudder contributions,
- use experiments such as turning circles and zig-zag tests,
- compare simulation against benchmark manoeuvres,
- identify hydrodynamic or empirical coefficients,
- improve the model structure when a purely numerical fit is insufficient.

The works listed in the references section use closely related ideas, especially:

- Fossen's marine craft formulation,
- MMG manoeuvring models,
- system identification of manoeuvring models from tests,
- physics-informed manoeuvring model refinement.

13. Final practical lessons from this Bluefin workflow

1. **A simple model is useful, but it will usually fail in turning.**
 2. **Sway matters.** If you ignore sway, turning fit is often poor.
 3. **A single drag term is rarely enough.**
 4. **Rudder force should not be treated as only a heading moment.**
 5. **If controller inputs are uncertain, fit to motion outputs instead.**
 6. **The pattern of error is more informative than a single score.**
 7. **Add complexity only when a specific mismatch justifies it.**
 8. **Keep the model usable.** A slightly simplified but interpretable model is often better than an unreadable full-physics port.
-

14. Current status

At the end of this workflow, the Bluefin simulator has moved from:

- a simple lumped thrust-drag-turn model,

to:

- a nonlinear 3-DOF MATLAB-inspired vessel model,
- with thrust shaping,
- separated rudder effects,
- and experimentally calibrated parameters.

The current v2 model is a practical **working baseline** for future simulation and RL work, even though it is not yet a perfect reproduction of the real vessel.

15. References and recommended reading

Core marine-craft dynamics

1. **Fossen, T. I.** *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed., Wiley, 2021.
Useful for the general theory of marine craft dynamics, guidance, navigation, and control.
2. **Fossen, T. I.** “Fossen’s Marine Craft Model.”
A concise online summary of the matrix–vector form of marine craft equations of motion.

Component-based ship manoeuvring models

3. **Yasukawa, H., and Yoshimura, Y.** “Introduction of MMG standard method for ship maneuvering predictions.” *Journal of Marine Science and Technology*, 2015.
Very relevant for the hull-propeller-rudder decomposition used in practical manoeuvring models.
4. **Ogawa, A., and Kasai, H.** “On the mathematical model of manoeuvring motion of ships.”
A classic reference behind component-based manoeuvring models.

System identification / fitting from tests

5. **Alexandersson, M., Mao, W., and Ringsberg, J. W.** “System identification of Vessel Manoeuvring Models.” *Ocean Engineering*, 2022.
Relevant for using inverse-dynamics-style fitting and comparing turning predictions to benchmark manoeuvres.
6. **Alexandersson, M., Mao, W., Ringsberg, J. W., and Kjellberg, M.** “System identification of a physics-informed ship model for better predictions in wind conditions.” *Ocean Engineering*, 2024.
Especially relevant because it adds a more physically informed rudder model to improve fit and generalization.

Test procedures and validation

7. **ITTC Manoeuvring Committee Report.**
Useful for understanding why turning circles and zig-zag tests are standard validation manoeuvres.

16. Suggested future work

The next logical improvements would be:

- further refining turning transients,
- improving the sustained yaw-rate match,
- validating against an additional manoeuvre beyond the two calibration tests,
- and introducing limited domain randomization for RL once the simulator is stable enough.